DarkOrchid Distributed Social Network API (1.0.0)

Download OpenAPI specification: Download

API documentation for CMPUT 404 Project: Distributed Social Network

authors

List all authors on this node (paginated)

When to use: Discover all authors hosted on this node, for federation or author directories.
Why use it: Other nodes use this to discover authors for federation.
How to use: Send a GET request to `/api/authors/`. Use `page` and `size` query params to paginate.
Why not use UI: This is the machine-readable/API version; use the browser authors list for human viewing.
Example: curl http://127.0.0.1:8000/api/authors?page=1&size=5

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

QUERY PARAMETERS

`page` integer
Examples:

- `page=2` - Page 2

Page number to retrieve.

`size` integer
Examples:

- `size=5` - 5 per page

Number of authors per page.

Responses

> **200** Paginated list of authors on this node.

— **400** Invalid `page` or `size` query parameter.

GET /api/authors/

Response samples

200

Content type
application/json

Copy Expand all Collapse all

```
{
    "type": "authors",
  - "authors": [
      + { … }
    ]
}
```

Get comments made by an author

When to use: Use this endpoint to list comments authored by one specific author.

```
                How to use:
                Send a GET request to `/api/authors/{author_id}/commented/`. Use `

                Why use it:
                This is the author-centric comment history route. It is useful for

                Why not use another route:
                Do not use the entry-comments route if your goal is to browse all

                Authentication:
                GET follows your node's local/remote access rules for author comme
```

```
                Pagination:
                - `page` (int): 1-based page number, default `1`
                - `size` (int): number of comments per page, default `10`

                GET response shape:
                Returns a `comments` object.
                - `type` (str): `comments`
                - `id` (str): FQID of the author's commented collection
                - `web` (str): frontend URL for the author
                - `page_number` (int): current page
                - `size` (int): page size
                - `count` (int): total number of comments by this author known to
                - `src` (array): current page of `comment` objects
```

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`author_id`
required

string
UUID of the author whose comments are requested.

QUERY PARAMETERS

`page`

integer
Default: `1`
Page number for pagination.

`size`

integer
Default: `10`
Number of comments per page.

Responses

> **200** Comments retrieved successfully.

> **404** Author not found.

GET `/api/authors/{author_id}/commented/`

Response samples

200 404

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "comments",
    "page_number": 1,
    "size": 5,
    "count": 1,
  - "src": [
      + { … }
    ]
}
```

Create a comment

When to use: Use this endpoint to create a new comment as a local author.

```
How to use:
Send a POST request to `/api/authors/{author_id}/commented/` with

Why use it:
Use this route when a local API client wants to create a comment a

Why not use another route:
Do not post here as a different logged-in user. Do not use this ro

Authentication:
POST requires local authentication as the same author.

Request fields:
- `type` (str, required): must be `comment`
- `entry` (str, required): FQID of the entry being commented on
- `comment` (str, required): text body of the comment
- `contentType` (str, optional): content format such as `text/plai

Response fields:
Returns a single `comment` object with:
- `type` (str): `comment`
- `author` (object): author who created the comment
- `comment` (str): comment body
- `contentType` (str): stored content format
```

```
                - `published` (str): ISO 8601 timestamp
                - `id` (str): comment FQID
                - `entry` (str): parent entry FQID
                - `web` (str, if present): frontend URL

                Common errors:
                - `400`: invalid JSON, missing required fields, or wrong `type`
                - `403`: requester is not the same local author
                - `404`: referenced entry could not be found
```

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

author_id required	string <uuid>

REQUEST BODY SCHEMA: application/json

type required	string
entry required	string
comment required	string
contentType	string

Responses

> **201** Comment created successfully.

— **400** Invalid request body.

— **403** Forbidden.

— **404** Entry not found.

POST /api/authors/{author_id}/commented/

Request samples

Payload

Content type
application/json

Copy

```
{
    "type": "comment",
    "entry": "http://nodebbbb/api/authors/222/entries/249",
    "comment": "Sick Olde English",
    "contentType": "text/markdown"
}
```

Response samples

201

Content type
application/json

Copy Expand all Collapse all

```
{
    "type": "comment",
  - "author": {
        "type": "author",
        "id": "http://nodeaaaa/api/authors/111",
        "displayName": "Greg Johnson"
    },
    "comment": "Sick Olde English",
    "contentType": "text/markdown",
    "published": "2015-03-09T13:07:04+00:00",
    "id": "http://nodeaaaa/api/authors/111/commented/130",
    "entry": "http://nodebbbb/api/authors/222/entries/249"
}
```

Get a specific comment made by an author

When to use: Use this endpoint to fetch one specific comment by local comment serial within one author's commented collection.

```
                How to use:
                Send a GET request to `/api/authors/{author_id}/commented/{comment

                Why use it:
                Use this route when you know both the author and the local comment

                Why not use another route:
                Do not use the FQID route if you already know the local author/com

                Authentication and visibility:
                Access follows your node's visibility rules for the referenced com

                Response shape:
                Returns a single `comment` object.
                - `type` (str): `comment`
                - `author` (object): the comment author
                - `comment` (str): body text
                - `contentType` (str): text format
                - `published` (str): ISO 8601 timestamp
                - `id` (str): fully qualified comment ID
                - `entry` (str): fully qualified parent entry ID
                - `web` (str): frontend URL
```

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`author_id` required	string UUID of the author who created the comment.
`comment_id` required	string UUID of the comment.

Responses

> **200** Comment retrieved successfully.

> **404** Author or comment not found.

GET /api/authors/{author_id}/commented/{comment_id}/

Response samples

200 404

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "comment",
  - "author": {
        "type": "author",
        "id": "http://nodeaaaa/api/authors/111",
        "web": "http://nodeaaaa/authors/greg",
        "host": "http://nodeaaaa/api/",
        "displayName": "Greg Johnson",
        "github": "http://github.com/gjohnson",
        "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
    },
    "comment": "Sick Olde English",
    "contentType": "text/markdown",
```

```
    "published": "2015-03-09T13:07:04+00:00",
    "id": "http://nodeaaaa/api/authors/111/commented/130",
    "entry": "http://nodebbbb/api/authors/222/entries/249",
    "web": "http://nodebbbb/authors/222/entries/249"
}
```

Get comments on an entry

Retrieves the comments on a specific entry.

Returns a Comments object as defined in the API specification.

Pagination parameters:

- `page` — page number (default 1)
- `size` — number of comments per page

Comments should be returned newest first.

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`author_id` required	string <uuid>
`post_id` required	string <uuid>

QUERY PARAMETERS

`page`	integer Default: `1` Page number for pagination.
`size`	integer Default: `5` Number of comments per page.

Responses

> **200** Comments retrieved successfully.

> **404** Entry not found.

GET /api/authors/{author_id}/entries/{post_id}/comments/

Response samples

200 404

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "comments",
    "id": "http://nodebbbb/api/authors/222/entries/249/comments",
    "web": "http://nodebbbb/authors/222/entries/249",
    "page_number": 1,
    "size": 5,
    "count": 2,
  - "src": [
      + { … }
    ]
}
```

Get a specific comment on an entry

When to use: Use this endpoint when the client knows the entry by fully qualified ID instead of local UUID path segments.

```
How to use:
Send a GET request to `/api/entries/{fqid}/comments/`, where `fqid
qualified entry ID. Use `page` and `size` for pagination. Page num

Why use it:
This route is useful for federation and remote-object workflows,
```

```
because entry IDs are globally unique FQIDs across nodes.

Why not use another route:
Do not use this route if you already have the local
`/api/authors/{author_id}/entries/{post_id}/comments/` route. The

Authentication and visibility:
This endpoint returns comments that this node knows
about for the requested entry, subject to the visibility of the pa

Pagination:
- `page` (int): 1-based page number, default `1`
- `size` (int): number of comments per page, default `5`

Response shape:
Returns a `comments` object with:
- `type` (str): `comments`
- `id` (str): FQID of the comments collection
- `web` (str): frontend URL for the parent entry
- `page_number` (integer): current page
- `size` (int): page size
- `count` (int): total comments known for this entry
- `src` (array): current page of `comment` objects

Comment object fields:
- `type` (str): `comment`
- `author` (object): the comment author
- `comment` (str): comment body
- `contentType` (str): body format such as `text/plain` or `text/m
- `published` (str): ISO 8601 timestamp
- `id` (str): comment FQID
- `entry` (str): parent entry FQID
- `web` (str): frontend URL
```

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`author_id` required	string UUID of the author who owns the entry.
`comment_fqid` required	string Fully qualified ID of the comment.
`post_id` required	string UUID of the entry.

Responses

> **200** Comment retrieved successfully.

> **404** Entry or comment not found.

GET /api/authors/{author_id}/entries/{post_id}/comments/{comment_fqid}/

Response samples

200 404

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "comment",
  - "author": {
        "type": "author",
        "id": "http://nodeaaaa/api/authors/111",
        "web": "http://nodeaaaa/authors/greg",
        "host": "http://nodeaaaa/api/",
        "displayName": "Greg Johnson",
        "github": "http://github.com/gjohnson",
        "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
    },
    "comment": "Sick Olde English",
    "contentType": "text/markdown",
    "published": "2015-03-09T13:07:04+00:00",
    "id": "http://nodeaaaa/api/authors/111/commented/130",
    "entry": "http://nodebbbb/api/authors/222/entries/249"
}
```


List incoming follow requests

```
**When to use:**  Use this endpoint when you want to see which authors are request
**Why use it:**  This allows the user to approve or deny follow requests and manag
**How to use it:**  Send a GET request to `/api/authors/{author_id}/follow_request

**Response:**  Returns a JSON object with a `type` of `"follow_requests"` and an `
```

AUTHORIZATIONS: > *cookieAuth* or *basicAuth*

PATH PARAMETERS

`author_id` required	string <uuid>

Responses

> **200** List of follow requests retrieved successfully.

— **403** Forbidden. Only the logged in author may view their incoming follow requests.

— **404** Author not found.

GET /api/authors/{author_id}/follow_requests/

Response samples

200

Content type
application/json

Copy Expand all Collapse all

```
{
  "type": "follow_requests",
- "items": [
    + { … }
  ]
```

```
}
```

Get visible stream for a specific author

```
    Gets the visible stream (posts) for a specific author, sorted newest-first (by

    When to use: Use this endpoint to display the inbox of posts an author is allo
    How to use: Send a GET request with the author's UUID.
    Pagination: Use query parameters: ?page=<number>&size=<number>

    Example usage (curl):
    curl -u username:password -X GET "http://127.0.0.1:8000/api/authors/<author_id
```

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

author_id
required

string <uuid>

Examples:

- `c7d84ea2-27bb-44ba-8e9c-07bc149a88ee` - Author UUID

UUID of the author whose visible stream is requested

QUERY PARAMETERS

page

integer

Examples:

- `page=1` - Page number

Page number (1-indexed) for pagination

size

integer

Examples:

- `size=10` - Page size

Number of posts per page for pagination

Responses

> **200** Paginated list of posts in the author's visible stream

— **400** Invalid page or size

— **403** You must be logged in to view this stream.

— **404** Author not found.

GET /api/authors/{author_id}/stream

Response samples

200

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "posts",
    "page": 1,
    "page_size": 10,
    "total_posts": 25,
  - "items": [
      + { … },
      + { … }
    ]
}
```

Get a single author by fqid

GET: Returns a single author's public profile as JSON.

```
When to use GET: When you need to look up a specific author's identity info by fqi

How to use GET: Send a GET request with the author's fqid in the URL. Authenticati

Pagination: Not applicable — returns a single object.

Example: curl -X GET http://127.0.0.1:8000/api/authors/http://127.0.0.1:8000/api/a
```

AUTHORIZATIONS: > *cookieAuth* or *basicAuth* or None

PATH PARAMETERS

`fqid` required	string

Responses

> **200** Author found and returned.

— **404** Author not found.

GET /api/authors/{fqid}/

Response samples

200

Content type
application/json

Copy


```
{
    "type": "author",
    "id": "http://127.0.0.1:8000/api/authors/c7d84ea2-27bb-44ba-8e9c-07bc149a88e
    "host": "http://127.0.0.1:8000/api/",
    "displayName": "Cristy",
```

```
    "github": "https://github.com/PinguinoF78",
    "profileImage": "https://picsum.photos/200",
    "web": "http://127.0.0.1:8000/authors/c7d84ea2-27bb-44ba-8e9c-07bc149a88ee"
}
```

Get things liked by an author (FQID)

Retrieves a list of things liked by the specified author, identified by their fully-qualified ID (FQID).

- Returns local and remote likes.
- Likes are ordered newest first.
- Pagination supported via `page` and `size` query parameters.
- Top-level likes object includes `id` (API URL) and `web` (HTML URL).

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`fqid` required	string

Responses

> **200** Likes object or a single like object

— **404** Author not found

GET /api/authors/{fqid}/liked

Response samples

200

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "likes",
    "id": "http://nodeaaaa/api/authors/111/liked",
    "web": "http://nodeaaaa/authors/111/liked",
    "page_number": 1,
    "size": 2,
    "count": 2,
  - "src": [
      + { … }
    ]
}
```

Get things liked by an author (FQID)

Retrieves a list of things liked by the specified author, identified by their fully-qualified ID (FQID).

- Returns local and remote likes.
- Likes are ordered newest first.
- Pagination supported via `page` and `size` query parameters.
- Top-level likes object includes `id` (API URL) and `web` (HTML URL).

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`fqid` required	string

Responses

> **200** Likes object or a single like object

— **404** Author not found

GET /api/authors/{fqid}/liked/

Response samples

200

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "likes",
    "id": "http://nodeaaaa/api/authors/111/liked",
    "web": "http://nodeaaaa/authors/111/liked",
    "page_number": 1,
    "size": 2,
    "count": 2,
  - "src": [
      + { … }
    ]
}
```

Get a single author by ID

GET: Returns a single author's public profile as JSON.

```
When to use GET: When you need to look up a specific author's identity info by UUI

How to use GET: Send a GET request with the author's UUID in the URL. No authentic

Pagination: Not applicable — returns a single object.
```

```
Example: curl -X GET http://127.0.0.1:8000/api/authors/c7d84ea2-27bb-44ba-8e9c-07b
```

AUTHORIZATIONS: > *cookieAuth* or *basicAuth* or None

PATH PARAMETERS

id required	string <uuid>

Responses

> **200** Author found and returned.

— **404** Author not found.

GET /api/authors/{id}/

Response samples

200

Content type
application/json

Copy

```
{
    "type": "author",
    "id": "http://127.0.0.1:8000/api/authors/c7d84ea2-27bb-44ba-8e9c-07bc149a88e
    "host": "http://127.0.0.1:8000/api/",
    "displayName": "Cristy",
    "github": "https://github.com/PinguinoF78",
    "profileImage": "https://picsum.photos/200",
    "web": "http://127.0.0.1:8000/authors/c7d84ea2-27bb-44ba-8e9c-07bc149a88ee"
}
```

Update a single author by ID

PUT: Updates the author's profile fields.

```
When to use PUT: When an author wants o update their displayName, github, or profi

How to use PUT: Send a PUT request with a JSON body containing the fields to updat

Why NOT to use PUT: Do not use PUT to change the author's UUID or host — these are

Pagination: Not applicable — returns a single object.

Example:
curl -X PUT http://127.0.0.1:8000/api/authors/c7d84ea2-27bb-44ba-8e9c-07bc149a88ee
       "displayName": "Mysterio",
       "github": "http://github.com/mysterio",
       "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
     }'       -u <username>:<password>
```

AUTHORIZATIONS: > *cookieAuth* or *basicAuth* or None

PATH PARAMETERS

`id` required	string <uuid>

REQUEST BODY SCHEMA: application/json

`displayName`	string The display name of the author. Can be updated.

`github`	string GitHub profile URL of the author. Can be updated.
`profileImage`	string URL of the author profile image. Can be updated.

Responses

> **200** Author updated successfully.

— **403** You are not authorized to update this author.

— **404** Author not found.

PUT `/api/authors/{id}/`

Request samples

Payload

Content type
application/json

Copy

```
{
    "displayName": "Mysterio",
    "github": "http://github.com/mysterio",
    "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
}
```

Response samples

200

Content type
application/json

Copy


```
{
    "type": "author",
    "id": "http://127.0.0.1:8000/api/authors/c7d84ea2-27bb-44ba-8e9c-07bc149a88e
    "host": "http://127.0.0.1:8000/api/",
    "displayName": "Jane Doe",
    "github": "http://github.com/janedoe",
    "profileImage": "https://i.imgur.com/k7XVwpB.jpeg",
    "web": "http://127.0.0.1:8000/authors/c7d84ea2-27bb-44ba-8e9c-07bc149a88ee"
}
```

List all entries for an author

When to use: When you need to retrieve all visible entries for a specific author.
Why use it: Returns entries filtered by your relationship to the author — public only, or more depending on if you follow or are friends with them.
How to use: Send a GET request to `/api/authors/<uuid>/entries/`. Authentication optional but affects visibility.
Why not use UI: This is the machine-readable/API version; use the browser post list for human viewing.

AUTHORIZATIONS: > *cookieAuth* or *basicAuth* or None

PATH PARAMETERS

`id` required	string Examples: • `c7d84ea2-27bb-44ba-8e9c-07bc149a88ee` - Valid UUID example The UUID of the author whose entries you want to list.

Responses

> **200** List of entries visible to the requester.

GET /api/authors/{id}/entries/

Response samples

200

Content type
application/json

Copy Expand all Collapse all

```
[
 - {
      "type": "entries",
    + "entries": [ … ]
   }
]
```

Create a new entry for an author

When to use: When you need to programmatically create an entry for an author.
Why use it: Allows automated or federated clients to publish entries without using the browser UI.
How to use: Send a POST request to `/api/authors/<uuid>/entries/` with title, content, visibility, and type in the body.
Why not use for edits/deletes: Use PUT or DELETE on `/api/authors/<uuid>/entries/<entry_id>/` instead.

AUTHORIZATIONS: > *cookieAuth* or *basicAuth* or None

PATH PARAMETERS

id required	string Examples: • `c7d84ea2-27bb-44ba-8e9c-07bc149a88ee` - Valid UUID example The UUID of the author creating the entry.

REQUEST BODY SCHEMA: application/json

property name* additional property	any

Responses

— **201** Entry created successfully.

— **400** Missing required fields.

POST /api/authors/{id}/entries/

Request samples

Payload

Content type
application/json

Example
Curl request example

Copy

```
"curl -X POST http://127.0.0.1:8000/api/authors/c7d84ea2-27bb-44ba-8e9c-07bc149a
```

Get a specific entry

Retrieves full details of a single entry. Visibility rules apply: FRIENDS entries only visible to friends, UNLISTED to anyone with the link.

```
Pagination: Not applicable — single entry.
```

AUTHORIZATIONS: > *cookieAuth* or *basicAuth* or None

PATH PARAMETERS

id required	string <uuid>
post_id required	string <uuid>

Responses

> **200** Entry retrieved successfully.

— **403** You are not authorized to view this entry.

— **404** Entry not found or already deleted.

GET /api/authors/{id}/entries/{post_id}/

Response samples

200

Content type
application/json

Copy


```
{
    "type": "entry",
    "id": "http://127.0.0.1:8000/api/authors/c7d84ea2-27bb-44ba-8e9c-07bc149a88e
    "title": "Hello World",
    "content": "This is my first entry to the world :D",
    "contentType": "text/plain",
    "visibility": "PUBLIC",
    "author": "http://127.0.0.1:8000/api/authors/c7d84ea2-27bb-44ba-8e9c-07bc149
    "created_at": "2026-02-28T12:00:00Z"
```

```
}
```

Edit a specific entry

Edits an existing entry. Only the author can edit their own entry. Send a JSON body with fields to update. Can update title, content, and visibility.

```
Why NOT to use PUT to delete: Use DELETE instead.

Pagination: Not applicable — single entry.
```

AUTHORIZATIONS: > *cookieAuth* or *basicAuth* or None

PATH PARAMETERS

`id` required	string <uuid>
`post_id` required	string <uuid>

REQUEST BODY SCHEMA: application/json

`title`	string
`content`	string
`visibility`	string

Responses

> **200** Entry updated successfully.

— **403** You are not authorized to edit this entry.

— **404** Entry not found or already deleted.

PUT /api/authors/{id}/entries/{post_id}/

Request samples

Payload

Content type
application/json

Copy

```
{
    "title": "Updated Title",
    "content": "Updated content.",
    "visibility": "PUBLIC"
}
```

Response samples

200

Content type
application/json

Copy

```
{
    "message": "Entry updated",
    "id": "a1b2c3d4-0000-0000-0000-000000000000"
}
```

Delete a specific entry

Soft-deletes an entry. Entry remains in database with is_deleted=True. Removed from all inboxes and hidden from UI and API. Cannot be undone via API. Node admins can still see deleted entries.

`Pagination: Not applicable — single entry.`

AUTHORIZATIONS: > *cookieAuth* or *basicAuth* or None

PATH PARAMETERS

`id` required	string <uuid>
`post_id` required	string <uuid>

Responses

> **200** Entry deleted successfully.

— **403** You are not authorized to delete this entry.

— **404** Entry not found or already deleted.

DELETE `/api/authors/{id}/entries/{post_id}/`

Response samples

200

Content type
application/json

Copy

```
{
    "message": "Entry deleted"
}
```

Get image entry as binary (specific author)

Returns the image content of an entry as binary data instead of base64.

```
When to use: Use this to display an image entry directly in an HTML img tag.

How to use: Use the URL directly as the src of an img tag.

Why to use: Allows markdown entries to embed images hosted on this node.

Why NOT to use: Do not use this for non-image entries — returns 404.

Pagination: Not applicable.

Example usage (curl):
curl -X GET "http://127.0.0.1:8000/api/authors/{author_id}/entries/{entry_id}/imag
```

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

id required	string <uuid>
post_id required	string <uuid>

Responses

> **200** Binary image data.

— **404** Post is not an image.

GET /api/authors/{id}/entries/{post_id}/image

Response samples

200

Content type
application/json

Copy

```
"HTTP/1.1 200 OK Content-Type: image/png Content-Length: 54231 (binary image dat
```

Get likes on a specific entry

Retrieves a paginated list of likes on the specified entry.

Visibility rules:

- PUBLIC entries: anyone can access.
- UNLISTED entries: anyone with the link.
- FRIENDS entries: only friends or the author can access.

Pagination is applied via query parameters `page` and `size`.

Returns a **Likes object** as defined in the API specification.

AUTHORIZATIONS: > *cookieAuth* or *basicAuth* or None

PATH PARAMETERS

`id` required	string UUID of the author who owns the entry.
`post_id` required	string UUID of the entry.

QUERY PARAMETERS

`page`	integer Page number for pagination (default: 1).
`size`	integer Number of likes per page (default: 50).

Responses

> **200** List of likes retrieved successfully.

— **403** Not authorized to view likes on this entry.

— **404** Entry not found or deleted.

GET /api/authors/{id}/entries/{post_id}/likes/

Response samples

200

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "likes",
    "id": "http://nodeaaaa/api/authors/111/entries/222/likes",
    "page_number": 1,
    "size": 50,
    "count": 2,
  - "src": [
      + { … }
    ]
}
```

Get a list of the selected author's followers

When to use: When the author wants a list of followers.
Why use it: Allows the use of third-party ui while communicating with our server. **How to use**: Send GET "/api/authors/{id}/followers/".

AUTHORIZATIONS: > *cookieAuth* or *basicAuth*

PATH PARAMETERS

id required	string <uuid>

Responses

> **200** Operation successfull

— **403** You are not authorized

— **404** Author does not exist

GET /api/authors/{id}/followers/

Response samples

200

Content type
application/json

Copy Expand all Collapse all

```
[
 - {
      "type": "followers",
    + "followers": [ … ]
   }
]
```

Check if an author in followers

When to use: When the author wants check if another author is a follower.
Why use it: Allows the use of third-party ui while communicating with our server and communication between node servers. **How to use**: Send GET to "/api/authors/{id}/followers/{fqid}/".

AUTHORIZATIONS: > *cookieAuth* or *basicAuth*

PATH PARAMETERS

`fqid` required	string
`id` required	string <uuid>

Responses

> **200** fqid author is a follower

— **403** You are not authorized

— **404** fqid author is not in followers

GET `/api/authors/{id}/followers/{fqid}`

Response samples

200

Content type
application/json

Copy


```
{
    "type": "author",
    "id": "http://darkorchid/api/authors/0a083be8-e6ca-458e-a0b0-3abf5e012aeb",
    "host": "http://darkorchid/api/",
    "displayName": "Amy",
    "github": "http://github.com/amy",
    "profileImage": "http://darkorchid/api/authors/0a083be8-e6ca-458e-a0b0-3abf5
```

```
    "web": "http://darkorchid/authors/0a083be8-e6ca-458e-a0b0-3abf5e012aeb"
}
```

To accept follow request

When to use: When the author wants to accept a follow request.
Why use it: Allows the use of third-party ui while communicating with our server. **How to use**: Send PUT to "/api/authors/{id}/followers/{fqid}/".

AUTHORIZATIONS: > *cookieAuth* or *basicAuth*

PATH PARAMETERS

`fqid` required	string
`id` required	string <uuid>

Responses

— **200** Successfully accepted fqid author's request

— **403** You are not authorized

— **404** fqid author has not requested to follow

PUT `/api/authors/{id}/followers/{fqid}`

decline follow request/remove follower

When to use: When the author wants to deny follow request or remove another author from followers.
Why use it: Allows the use of third-party ui while communicating with our server. **How to use**: Send DELETE to "/api/authors/{id}/followers/{fqid}/".

AUTHORIZATIONS: > *cookieAuth* or *basicAuth*

PATH PARAMETERS

`fqid` required	string
`id` required	string <uuid>

Responses

— **200** Successfully removed fqid author form followers and/or request their request

— **403** You are not authorized

— **404** fqid author is not a follower and has not requested

DELETE `/api/authors/{id}/followers/{fqid}`

Get a list authors that the selected author is following

When to use: When the author wants a list of authors they are following.
Why use it: Allows the use of third-party ui while communicating with our server. **How to use**: Send GET "/api/authors/{id}/following/".

AUTHORIZATIONS: > *cookieAuth* or *basicAuth*

PATH PARAMETERS

id required	string <uuid>

Responses

> **200** Operation successfull

— **403** You are not authorized

— **404** Author does not exist

GET /api/authors/{id}/following/

Response samples

200

Content type
application/json

Copy Expand all Collapse all

```
[
 - {
      "type": "following",
    + "following": [ … ]
   }
]
```

Check if following author

When to use: When the author wants check if they are following another author.
Why use it: Allows the use of third-party ui while communicating with our server. **How to use**: Send GET to "/api/authors/{id}/following/{fqid}/".

AUTHORIZATIONS: > *cookieAuth* or *basicAuth*

PATH PARAMETERS

`fqid` required	string
`id` required	string <uuid>

Responses

> **200** Operation successfull

— **403** You are not authorized

— **404** Operation not successfull

GET `/api/authors/{id}/following/{fqid}`

Response samples

200

Content type
application/json

Copy

```
{
    "type": "author",
    "id": "http://darkorchid/api/authors/0a083be8-e6ca-458e-a0b0-3abf5e012aeb",
    "host": "http://darkorchid/api/",
    "displayName": "Amy",
    "github": "http://github.com/amy",
    "profileImage": "http://darkorchid/api/authors/0a083be8-e6ca-458e-a0b0-3abf5
    "web": "http://darkorchid/authors/0a083be8-e6ca-458e-a0b0-3abf5e012aeb"
}
```


---

send follow request

When to use: When the author wants follow (send request) to another author.
Why use it: Allows the use of third-party ui while communicating with our server. **How to use**: Send PUT to "/api/authors/{id}/following/{fqid}/".

AUTHORIZATIONS: > *cookieAuth* or *basicAuth*

PATH PARAMETERS

`fqid` required	string
`id` required	string <uuid>

Responses

— **200** Successfully sent follow request to fqid

— **403** You are not authorized

— **404** Problem with foreign node of fqid author

PUT `/api/authors/{id}/following/{fqid}`

unfollow an author

When to use: When the author wants to unfollow another author.
Why use it: Allows the use of third-party ui while communicating with our server. **How to use**: Send DELETE to "/api/authors/{id}/following/{fqid}/".

AUTHORIZATIONS: > *cookieAuth* or *basicAuth*

PATH PARAMETERS

`fqid` required	string
`id` required	string <uuid>

Responses

— **200** Successfully unfollowed fqid author or revoked follow request

— **403** You are not authorized

— **404** not following fqid author

DELETE `/api/authors/{id}/following/{fqid}`

Post a follow request or entry to inbox

When to use: When a node wants to inform about a post or friend request.
Why use it: This is the standard, federated way to create posts.
How to use: POST to `/api/authors/<id>/inbox/` with form data following example.

AUTHORIZATIONS: > *cookieAuth* or *basicAuth* or None

PATH PARAMETERS

id required	string <uuid>

REQUEST BODY SCHEMA: application/json

property name* additional property	any

Responses

> **200** Follow request or entry saved successfully

> **400** The object author does not match the inbox author

— **403** Not authorized

— **404** Inbox's author not found

POST /api/authors/{id}/inbox

Request samples

Payload

Content type
application/json

Example
Follow request body

Copy Expand all Collapse all

```
{
    "type": "follow",
  - "actor": {
        "type": "author",
        "id": "http://darkorchid/api/authors/<id>",
        "host": "http://darkorchid/api/",
        "displayName": "Amy",
        "web": "http://darkorchid/authors/<id>",
        "github": "http://github.com/amy",
        "profileImage": "http://darkorchid/api/authors/<id>/entries/<post_id>/ima
    },
  - "object": {
        "type": "author",
        "id": "http://darkorchid/api/authors/<id>",
        "host": "http://darkorchid/api/",
        "displayName": "Jane",
        "web": "http://darkorchid/authors/<id>",
        "github": "http://github.com/jane",
        "profileImage": "http://darkorchid/api/authors/<id>/entries/<post_id>/ima
    }
}
```


Response samples

200 400

Content type
application/json

Example
Follow request response

Copy


```
{
    "message": "Friend request added"
}
```


Get things liked by an author

Retrieves a list of things liked by the specified author, or a single like if `like_serial` is provided.

- Returns local and remote likes.
- Likes are ordered newest first.
- Pagination supported via `page` and `size` query parameters.
- Top-level likes object includes `id` (API URL) and `web` (HTML URL).

AUTHORIZATIONS: > *cookieAuth* or *basicAuth*

PATH PARAMETERS

`id` required	string <uuid>

Responses

> **200** A likes object or a single like object

— **403** Not authorized

— **404** Author or like not found

GET `/api/authors/{id}/liked/`

Response samples

200

Content type
application/json

Example
List of likes

Copy Expand all Collapse all

```
{
    "type": "likes",
    "id": "http://nodeaaaa/api/authors/111/liked",
    "web": "http://nodeaaaa/authors/111/liked",
```

```
    "page_number": 1,
    "size": 2,
    "count": 2,
  - "src": [
      + { … },
      + { … }
    ]
}
```

Get things liked by an author

Retrieves a list of things liked by the specified author, or a single like if `like_serial` is provided.

- Returns local and remote likes.
- Likes are ordered newest first.
- Pagination supported via `page` and `size` query parameters.
- Top-level likes object includes `id` (API URL) and `web` (HTML URL).

AUTHORIZATIONS: > *cookieAuth* or *basicAuth*

PATH PARAMETERS

`id` required	string <uuid>
`like_serial` required	string

Responses

> **200** A likes object or a single like object

— **403** Not authorized

— **404** Author or like not found

GET /api/authors/{id}/liked/{like_serial}/

Response samples

200

Content type
application/json

Example
List of likes

Copy Expand all Collapse all


```
{
    "type": "likes",
    "id": "http://nodeaaaa/api/authors/111/liked",
    "web": "http://nodeaaaa/authors/111/liked",
    "page_number": 1,
    "size": 2,
    "count": 2,
  - "src": [
      + { … },
      + { … }
    ]
}
```


---

commented

Get a specific comment by FQID

When to use: Use this endpoint when the only identifier you have for a comment is its fully qualified ID.

```
How to use:
Send a GET request to `/api/commented/{comment_fqid}/`, where `com

Why use it:
This route is useful for federation, remote references, bookmarks,

Why not use another route:
Do not use this route if you already know the local `/api/authors/

Authentication and visibility:
Access follows the visibility rules of the parent entry and the lo

Response shape:
Returns a single `comment` object.
- `type` (string): `comment`
- `author` (object): author who wrote the comment
- `comment` (string): body text
- `contentType` (string): body format such as `text/plain` or `tex
- `published` (string): ISO 8601 timestamp
- `id` (string): fully qualified comment ID
- `entry` (string): fully qualified parent entry ID
- `web` (string): frontend URL
```

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`comment_fqid` required	string Fully qualified ID (FQID) of the comment. URL-encoded if needed.

Responses

> **200** Comment retrieved successfully.

— **404** Comment not found.

GET `/api/commented/{comment_fqid}/`

Response samples

200

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "comment",
  - "author": {
        "type": "author",
        "id": "http://nodeaaaa/api/authors/111",
        "web": "http://nodeaaaa/authors/greg",
        "host": "http://nodeaaaa/api/",
        "displayName": "Greg Johnson",
        "github": "http://github.com/gjohnson",
        "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
    },
    "comment": "Sick Olde English",
    "contentType": "text/markdown",
    "published": "2015-03-09T13:07:04+00:00",
    "id": "http://nodeaaaa/api/authors/111/commented/130",
    "entry": "http://nodebbbb/api/authors/222/entries/249",
    "web": "http://nodebbbb/authors/222/entries/249"
}
```


---

entries

Get comments for an entry by FQID

When to use: Use this endpoint when a client needs the comment thread for a specific local entry using the local author UUID and entry UUID route.

```
                How to use:
                Send a GET request to `/api/authors/{author_id}/entries/{post_id}/

                Why use it:
```

```
Use this route when you already know the local author serial and l

Why not use another route:
We do not use the FQID route if we already have the local serial r

Authentication and visibility:
The response is visibility limited by the parent entry. Public and

Pagination:
- `page` (int): 1-based page number, default `1`
- `size` (int): number of comments per page, default `5`

Response shape:
Returns a `comments` object.
- `type` (str): always `comments`
- `id` (str): FQID of this comments collection
- `web` (str): frontend URL for the parent entry
- `page_number` (int): current page number
- `size` (int): page size
- `count` (int): total number of comments known for this entry
- `src` (array): current page of comment objects

Each item in `src` is a `comment` object.
- `type` (str): always `comment`
- `author` (object): author who wrote the comment
- `comment` (str): text body of the comment
- `contentType` (str): format such as `text/plain` or `text/markdo
- `published` (str): ISO 8601 timestamp
- `id` (str): FQID of the comment
- `entry` (str): FQID of the parent entry
- `web` (str): frontend URL related to the comment or entry page
- `likes` (object, if present): likes summary for the comment
```

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`fqid` required — string
Fully qualified ID of the entry.

QUERY PARAMETERS

`page` — integer
Default: `1`
Page number.

`size` — integer
Default: `5`
Number of comments per page.

Responses

> **200** Comments retrieved successfully.

> **404** Entry not found.

GET /api/entries/{fqid}/comments

Response samples

200 404

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "comments",
    "id": "http://nodebbbb/api/authors/222/entries/249/comments",
    "web": "http://nodebbbb/authors/222/entries/249",
    "page_number": 1,
    "size": 5,
    "count": 1,
  - "src": [
      + { … }
    ]
}
```

Get comments for an entry by FQID

When to use: Use this endpoint when a client needs the comment thread for a specific local entry using the local author UUID and entry UUID route.

```
How to use:
Send a GET request to `/api/authors/{author_id}/entries/{post_id}/

Why use it:
Use this route when you already know the local author serial and l

Why not use another route:
We do not use the FQID route if we already have the local serial r

Authentication and visibility:
The response is visibility limited by the parent entry. Public and

Pagination:
- `page` (int): 1-based page number, default `1`
- `size` (int): number of comments per page, default `5`

Response shape:
Returns a `comments` object.
- `type` (str): always `comments`
- `id` (str): FQID of this comments collection
- `web` (str): frontend URL for the parent entry
- `page_number` (int): current page number
- `size` (int): page size
- `count` (int): total number of comments known for this entry
- `src` (array): current page of comment objects

Each item in `src` is a `comment` object.
- `type` (str): always `comment`
- `author` (object): author who wrote the comment
- `comment` (str): text body of the comment
- `contentType` (str): format such as `text/plain` or `text/markdo
- `published` (str): ISO 8601 timestamp
- `id` (str): FQID of the comment
- `entry` (str): FQID of the parent entry
- `web` (str): frontend URL related to the comment or entry page
- `likes` (object, if present): likes summary for the comment
```

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`fqid` required	string Fully qualified ID of the entry.

QUERY PARAMETERS

`page`	integer Default: `1` Page number.

`size`	integer Default: `5` Number of comments per page.

Responses

> **200** Comments retrieved successfully.

> **404** Entry not found.

GET `/api/entries/{fqid}/comments/`

Response samples

200 404

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "comments",
    "id": "http://nodebbbb/api/authors/222/entries/249/comments",
    "web": "http://nodebbbb/authors/222/entries/249",
    "page_number": 1,
    "size": 5,
    "count": 1,
  - "src": [
      + { … }
    ]
}
```


Get image entry as binary

Returns the image content of an entry as binary data instead of base64.

```
When to use: Use this to display an image entry directly in an HTML img tag.

How to use: Use the URL directly as the src of an img tag.

Why to use: Allows markdown entries to embed images hosted on this node.

Why NOT to use: Do not use this for non-image entries — returns 404.

Pagination: Not applicable.

Example usage (curl):
curl -X GET "http://127.0.0.1:8000/api/entries/<fqid>/image/" -u username:password
```

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

fqid required	string

Responses

> **200** Binary image data.

— **404** Post is not an image.

GET /api/entries/{fqid}/image

Response samples

200

Content type
application/json

Copy

```
"HTTP/1.1 200 OK Content-Type: image/png Content-Length: 54231 (binary image dat
```

Get likes on an entry by FQID

Retrieves a paginated list of likes on the specified entry, identified by its fully-qualified ID (FQID).

This endpoint is equivalent to `GET /authors/{AUTHOR}/entries/{ENTRY}/likes` but allows fetching likes using the FQID.

Visibility rules:

- PUBLIC: anyone can view
- UNLISTED: anyone with the link
- FRIENDS: only friends or the author can view

Pagination is applied via query parameters `page` and `size`.

Returns a **Likes object** as defined in the API specification.

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`fqid` required	string Fully-qualified ID (FQID) of the entry (URL-encoded if needed).

QUERY PARAMETERS

`page`	integer Page number for pagination (default: 1).
`size`	integer Number of likes per page (default: 50).

Responses

> **200** Likes retrieved successfully.

— **403** Not authorized to view likes on this entry.

— **404** Entry not found.

GET /api/entries/{fqid}/likes

Response samples

200

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "likes",
    "id": "http://nodeaaaa/api/authors/111/entries/222/likes",
    "page_number": 1,
    "size": 50,
    "count": 2,
  - "src": [
      + { … }
    ]
}
```

Get likes on an entry by FQID

Retrieves a paginated list of likes on the specified entry, identified by its fully-qualified ID (FQID).

This endpoint is equivalent to `GET /authors/{AUTHOR}/entries/{ENTRY}/likes` but allows fetching likes using the FQID.

Visibility rules:

- PUBLIC: anyone can view
- UNLISTED: anyone with the link
- FRIENDS: only friends or the author can view

Pagination is applied via query parameters `page` and `size`.

Returns a **Likes object** as defined in the API specification.

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`fqid` required	string Fully-qualified ID (FQID) of the entry (URL-encoded if needed).

QUERY PARAMETERS

`page`	integer Page number for pagination (default: 1).
`size`	integer Number of likes per page (default: 50).

Responses

> **200** Likes retrieved successfully.

— **403** Not authorized to view likes on this entry.

— **404** Entry not found.

GET `/api/entries/{fqid}/likes/`

Response samples

200

Content type
application/json

Copy Expand all Collapse all

```
{
    "type": "likes",
    "id": "http://nodeaaaa/api/authors/111/entries/222/likes",
    "page_number": 1,
    "size": 50,
    "count": 2,
  - "src": [
      + { … }
    ]
}
```


---

liked

Get a single like by FQID

Retrieves a single like object by its fully-qualified ID (FQID).

- FQID must match the `id` field of the like exactly.
- Returns the full like object, including author info, object liked, and timestamp.

AUTHORIZATIONS: > *basicAuth* or *cookieAuth* or None

PATH PARAMETERS

`fqid` required	string

Responses

> **200** Single like object

— **404** Like not found

GET /api/liked/{fqid}

Response samples

200

Content type
application/json

Copy Expand all Collapse all


```
{
    "type": "like",
    "id": "http://nodeaaaa/api/authors/111/liked/166",
  - "author": {
        "type": "author",
        "id": "http://nodeaaaa/api/authors/111",
        "host": "http://nodeaaaa/api/",
        "web": "http://nodeaaaa/authors/111",
        "displayName": "Greg Johnson",
        "github": "http://github.com/gjohnson",
        "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
    },
    "published": "2015-03-09T13:07:04+00:00",
    "object": "http://nodebbbb/api/authors/222/entries/249"
}
```